Документация к библиотеке QComputations

Содержание

1 Структура библиотеки **3**

2 Прототипы функций и классов **6**

2.1 Обычная матрица Matrix<T> 6
2.2 Блочная матрица BLOCKED_Matrix<T> 9
2.3 BLOCKED_Vector<T> 12
2.4 Вспомогательные функции 13
2.5 Понятие БАЗОВОГО состояния Basis_State 17
2.6 Понятие состояния State<StateType> 19
2.7 Понятие оператора Operator<T>. 20
2.8 Понятие гамильтониана. 20
2.9 Моделирование динамики гамильтонианов. 20

3 Визуализация **20**

1 Структура библиотеки

Идея данной библиотеки заключается в том, чтобы полностью избавить программиста от рутины реализации, оптимизации и параллелизации при моделировании квантовых процессов. Сложность заключается в том, что каждый эксперимент и модель, по своей сути, уникальны. Разные структуры состояний систем, способы их создания и генерации, разные операторы, разные численные методы моделирования и способы паралеллизации. Данная библиотека унифицирует всё это, насколько это возможно к определённым базовым понятиям, с которыми потом и работает.

Для начала нужно разобраться со вспомогательными классами, на которых основывается вся библиотека. Многоядерная версия (CPU_CLUSTER) включает в себя всё, что входит и в одноядерную (SINGLE), то есть, является расширением последней, поэтому сначала начнём с того, что входит в них обоих.

- Основой всей библиотеки является класс **Matrix<T>**, который поддерживает типы **double** и **std::complex<double>**. Работа с ним упрощена настолько, насколько возможно. В нём реализованы все базовые операции и алгоритмы, часто используемые в различного рода вычислениях. Подробное описание данного класса описано в главе **Прототипы функций и классов**. Реализация написана в файлах (все исходные файлы лежат в директории src/QComputations/) matrix.*

- То, что я назвал "дополнительными операторами". Всякие удобные операторы, например умножение **std::vector<T>** на число типа **T**, умножение комплексной и вещественных матриц и так далее. Дополнительные операторы написаны в файлах additional_operators.*

- Дополнительные функции. Например, является ли число нулём или linspace (точная копия из языка Python, и часто используемый в программах).

- QConfig - конфигурационный синглтон (класс, который хранится в памяти в единственном экземпляре. Очень важная деталь библиотеки. Подробно описан в главе **QConfig**.)

Далее идут "основы который есть только в MPI версии.

Блочная матрица - класс **BLOCKED_Matrix<T>**, также реализованы типы **double** и **std::complex<double>**. Данный класс предназначен для упрощённой работы с пакетом Intel OneAPI (поэтому без него MPI версия не работает). Реализация в файлах blocked_matrix.* . Все прототипы функций, кроме конструкторов, аналогичны одноядерной версии. Сам принцип работы можно посмотреть в документации к Intel ScaLapack. Данный класс упрощает всю работу с библиотекой, благодаря ему, вообще не нужно думать о программе на уровне ядер. Вся параллелизация уже реализована в самом классе.

Дополнительные MPI функции, по своей сути, упрощает работу с часто используемыми функциями в Intel ScaLapack. Используются в реализации блочных матриц, и данный пакет довольно сложен в освоении, потому вообще не рекомендуется его трогать, но сама его реализация в файлах mpi_functions.*

Вообще, целью данной библиотеки является упрощение для программиста в принципе заниматься моделированием и вычислениями в данной сфере, поэтому "дополнительные операторы или, например, функции для MPI, скрываются "под капотом". Подразумевается, что Вы как раз и не должны думать о мудрёных Intel MKL и Intel ScaLapack, поэтому,

вообще, неплохо бы знать только классы матриц и вспомогательных функций (Не MPI), остальные - для продвинутого пользования.

Теперь разберём понятие, используемые в качестве абстракций для самих квантовых вычислений.

Всё начинается с понятия состояния, по своей сути, оно определяет всю систему. Базовым понятием состояния является класс **Basis_State**. Любой класс, являющийся наследником **Basis_State**, принимается библиотекой. Реализация в файлах state.* Далее, я буду использовать класс **StateType**, для обозначения подобного класса.

Сам класс **Basis_State** включает в себя 2 главных понятия:

1. Кудита - многозначного кубита, максимальное значение которого также хранится в классе в массиве **max_vals**.

2. Группа кудитов: во многих системах очень удобно разделение кудитов на группы, как с точки зрения индексации, так и с точки зрения математических нотаций. (В случае, когда оператор применяется к какой-то подсистеме, к остальным кудитам применяется единичный оператор)

Начальное же состояние, которое может быть суперпозицией нескольких состояний, создаётся с помощью шаблонного класса **State<StateType>**, то есть **StateType** - это какой-то элемент базиса, само же общепринятое понятие состояния задаётся через **State<StateTy**
Примеры можно посмотреть в директории библиотеки examples, подробно о них написано в инструкции. На основе данного состояния будет дальше генерироваться сам базис (данный процесс называется Отбор рабочей области (реализация в файлах graph.*)). **Внимание! На данный момент нет автоматической нормализации данного вектора. Проверки на дураков также нет.**

Следующее понятие, являющейся основой данной библиотеки: **Operator<StateType>**. Реализация в quantum_operators.* . Данный класс предназначен для создания собственных операторов и для их унификации. Работает это следующим образом. Вы пишите операторы в функциях следующего прототипа:

```
State<Some_State> some_op(const Some_State& state);
```

После этого, вы создаёте сам ваш оператор следующим образом:

```
using OpType = Operator<Some_State>;
OpType H = OpType(some_op_1_1) * OpType(some_op_1_2) * ... *
           OpType(some_op_1_n) +
           OpType(some_op_2_1) * OpType(some_op_2_2) * ... *
           OpType(some_op_2_n) +
           ...
           OpType(some_op_m_1) * OpType(some_op_m_2) * ... *
           OpType(some_op_m_n);
```

Данный оператор передаётся в конструктор гамильтониана. Можно также использовать напрямую для прогонки конкретного состояния через метод **apply**, ну или неявно просто умножить на **State<StateType>**.

Далее идёт само понятие гамильтониана. Класс-конструктор для гамильтониана от произвольного состояния: **H_by_Operator<StateType>** для одноядерной версии и **BLOCKED_H**

для многоядерной. В нём же генерируется базис. В библиотеке также есть и готовые гамильтонианы и понятия состояний, например **TCH_State** и **H_TCH**, которые описаны в главе **Описание готовых состояний, гамильтонианов и операторов**. Все эти классы являются наследниками класса **Hamiltonian** (для многоядерной версии **BLOCKED_Hamiltonia**
Классы родители всегда работают только с состоянием **Basis_State**, то есть, если вы вытащите базис из гамильтониана методом **get_basis**, вы получите **std::set<Basis_State>**. Так сделано по причине экономии памяти и в принципе по другому унифицировать эффективно чисто программно невозможно. (На данном языке уж точно)

Дальнейшее моделирование инвариантно относительно всех предыдущих понятий с точки зрения реализации и классов. Поэтому, моделирование происходит с помощью отдельных функций и методов, работающие лишь с **State<Basis_State>**, который сначала генерируется из базиса гамильтониана с помощью функции **fit_to_basis**, а также с **Hamiltonian**. Кастинг к **StateType** к **Basis_State** происходит автоматически. Примеры смотреть в examples.

На данный момент реализованы методы (Подробнее в главе **Методы моделирования**):

- Моделирование уравнения Шрёдингера через спектральное разложение. (**schrodinger**)
- Моделирование основного квантового уравнения с помощью метода Рунге-Кутта 2 и 4 порядков. (**quantum_master_equation**, выбор порядка метода происходит с помощью QConfig. (по умолчанию 2))

Результаты этих методов в дальнейшем визуализируется либо напрямую в самой библиотеке в версиях с Python API, либо через файловую систему и дополнительный скрипт, путь к которому будет храниться после установки в переменной окружения $SEABORN_PLOT. (Настройки к нему в переменной $SEABORN_CONFIG) Об этом подробно написано в главе **Визуализация**.

Для более детального понимания, приведена схема зависимостей внутри стуктуры данной библиотеки.

Библиотека, на данный момент, делится на 4 версии:

1. Версия на одном ядре с поддержкой графиков (libQComputations_SINGLE.so)
2. Версия на одном ядре без поддержки графиков (libQComputations_SINGLE_NO_PLOTS.so)
3. Версия на многих ядрах с поддержкой графиков (libQComputations_CPU_CLUSTER.so)
4. Версия на многих ядрах без поддержки графиков (libQComputations_CPU_CLUSTER_NO_

2 и 4 версии предполагают использование файловой системы для хранения результатов, они сохраняют в указанные программистом директории файлы в формате CSV. Визуализация этих данных происходит с помощью скрипта **seaborn_plot.py** на языке Python, путь к которому, после установки библиотеки, будет хранится в переменной окружения $SEABORN_PLOT. Инструкция использования данного скрипта находится в главе **Визуализация**.

Библиотека делится на 2 типа (на данный момент): последовательная/одноядерная (1, 2) и MPI (3, 4). В MPI также полностью входит одноядерная версия, поэтому можно комбинировать методы параллелизации между собой, например параллелить набор не блочных гамильтонианов по ним же самим.

2 Прототипы функций и классов

Здесь рассматриваются только сами классы и их методы. Саму детальную реализацию смотреть в исходный файлах (src/QComputations/).

В местах, где можно вставить тип **double** или **std::complex<double>**, будет написано **NumType** или **T**.

2.1 Обычная матрица Matrix<T>

matrix_style - хранения матрицы по строкам (Си стиль) или по столбцам (Фортрановский стиль). Файлы matrix.*

```
template<typename T> class Matrix {
    public:
        Matrix() = default;
        // Иницианилизирует матрицу размера n x m (n - число строк, m - число столбц
        explicit Matrix(MATRIX_STYLE matrix_style, size_t n, size_t m);

        // Иницианилизирует матрицу размера n x m с начальным значением во всех элем
        explicit Matrix(MATRIX_STYLE matrix_style, size_t n, size_t m, const T& init

        // Конструктор копирования
        explicit Matrix(const Matrix<T>& A);

        // Если у вас хранится одномерный массив, его можно привести к виду матрицу,
        explicit Matrix(const std::vector<T>& mass, size_t n, size_t m, MATRIX_STYLE

        // Создать матрицу, с помощью функции типа std::function<COMPLEX(size_t i, s
        explicit Matrix(MATRIX_STYLE matrix_style, size_t n, size_t m, std::function

        // Привести матрицу к другому типу (реализацию оставил для понимания)
        template<typename V>
        Matrix(const Matrix<V>& A): n_(A.n()), m_(A.m()) {
            for (size_t i = 0; i < n_; i++) {
                for (size_t j = 0; j < m_; j++) {
                    mass_.emplace_back(static_cast<V>(A[i][j]));
                }
            }

            matrix_style_ = A.get_matrix_style();
        }

        // Привести матрицу вида вектора векторов к нашему
        explicit Matrix(const std::vector<std::vector<T>>& A, MATRIX_STYLE matrix_st

        Matrix<T>& operator=(const Matrix<T>& A);
```

```
// Вместо строки под номером index вставить строку v
void modify_row(size_t index, const std::vector<T>& v);

// Вместо столбца под номером index вставить строку v
void modify_col(size_t index, const std::vector<T>& v);

std::vector<T> row (size_t index) const;
std::vector<T> col (size_t index) const;
size_t n() const { return n_; }
size_t size() const { return n_; } // удобно для квадратных матриц
size_t m() const { return m_; }

// Увеличить число строк в матрице
void add_rows(size_t row_count);

// Увеличить число столбцов в матрице
void add_cols(size_t col_count);

// Удалить число строк в матрице
void remove_rows(size_t row_count);

// Удалить число столбцов в матрице
void remove_cols(size_t col_count);

void expand(size_t n); // add_rows + add_cols
void reduce(size_t n); // remove_rows + remove_cols

// Не добавлять шаблонные виды операторов для матриц
Matrix<T> operator* (const Matrix<T>& A) const;
Matrix<T> operator+ (const Matrix<T>& A) const;
Matrix<T> operator- (const Matrix<T>& A) const;

Matrix<T>& operator+=(const Matrix<T>& A);
Matrix<T>& operator-=(const Matrix<T>& A);

std::vector<T> operator* (const std::vector<T>& v) const;

void operator*= (const T& num);

Matrix<T> operator* (const T& num) const;
Matrix<T> operator+ (const T& num) const;
Matrix<T> operator- (const T& num) const;
Matrix<T> operator/ (const T& num) const;
void operator/= (const T& num);

bool operator==(const Matrix<T>& A) const;
```

```
    // Вернуть матрицу в хранимом виде (в виде вектора)
    std::vector<T>& get_mass() { return mass_; }
    std::vector<T> get_mass() const { return mass_; }

    // Вернуть указатель на массив
    T* data() { return mass_.data(); }
    const T* data() const { return mass_.data(); }

    Matrix<T> transpose() const;
    Matrix<T> hermit() const;
    void show(size_t width = QConfig::instance().width()) const;

    // (!!!) Индексация для C стиля
    T* operator[](size_t index_row) { return mass_.data() + index_row * m_; };
    const T* operator[](size_t index_row) const { return mass_.data() + index_ro

    // (!!!) Инддексация для Фортрановского стиля
    T& operator()(size_t index_row, size_t index_col) { return mass_.data()[inde
    const T operator()(size_t index_row, size_t index_col) const { return mass_.

    // Лидирующее измерение
    size_t LD() const { return (matrix_style_ == C_STYLE ? m_ : n_); }

    bool is_c_style() const { return matrix_style_ == C_STYLE; }
    MATRIX_STYLE get_matrix_style() const { return matrix_style_; }
    void to_fortran_style();
    void to_c_style();

    // Вернуть подматрицу размера n x m, начало которой находится по координатам
    Matrix<T> submatrix(size_t n, size_t m, size_t row_index, size_t col_index)

    // Индексация общего вида, независимо от стиля матрицы
    size_t index(size_t i, size_t j) const { if (matrix_style_ == C_STYLE) retur
                                                else return j * n_ + i; }

    // Получить элемент матрицы, с помощью которого также можно менять матрицу
    T& elem(size_t i, size_t j) { return mass_.data()[this->index(i, j)]; }
    const T elem(size_t i, size_t j) const { return mass_.data()[this->index(i,

    // Записать матрицу в файл, в формате CSV. Точность регулируется с помощью Q
    void write_to_csv_file(const std::string& filename) const;
private:
    size_t n_; // Число строк
    size_t m_; // Число столбцов
    std::vector<T> mass_; // Массив с данными
```

```
        MATRIX_STYLE matrix_style_ = DEFAULT_MATRIX_STYLE; // Стиль матрицы - по умо
};
```

2.2 Блочная матрица BLOCKED_Matrix<T>

Блочная матрица напрямую связана с решёткой BLACS (придумка Intel OneAPI). Тип ILP_TYPE в библиотеке - это тип, зависящий от того, с помощью какой версий библиотек Intel OneAPI идёт компиляция, подробнее смотреть в самой документации Intel OneAPI. На данный момент библиотека компилируется с обычными библиотеками, и ILP_TYPE=int. Реализована вспомогательная функция для инициализации решётки BLACS - init_grid()

MATRIX_TYPE - это перечисляемый тип:

```
enum MATRIX_TYPE { GE = 120, SY = 121, HE = 122};
```

GE - обычная матрица. SY - симметричная. HE - эрмитовая. Как не покажется странным, в пакете Intel OneAPI нет экономии памяти на случай последних двух. Элементы нижней треугольной части просто не инициализируются. (Причём обязательно)

ПРЕДУПРЕЖДЕНИЕ. ПРИ НЕ КВАДРАТНЫХ РАЗМЕРАХ БЛОКОВ (NB, MB) НЕ РАБОТАЕТ ЭРМИТОВОЕ РАЗЛОЖЕНИЕ МАТРИЦЫ. ВЫДАЁТ НЕКОРРЕКТНУЮ ОШИБКУ.

```
template <typename T>
class BLOCKED_Matrix {
    public:
        explicit BLOCKED_Matrix() = default;

        // ??????????????
        explicit BLOCKED_Matrix(const BLOCKED_Matrix<T>& A, const Matrix<T>& local_m

        // Сгенерировать матрицу с помощью функции. NB и MB - размеры блоков (подбло
        // значения 0 означает выбор размера блока по умолчанию. (Рекомендуется)
        explicit BLOCKED_Matrix(ILP_TYPE ctxt, MATRIX_TYPE type, size_t n, size_t m,

        // Сгенерировать матрицу с начальным значанием value
        explicit BLOCKED_Matrix(ILP_TYPE ctxt, MATRIX_TYPE type, size_t n, size_t m,

        // Сгенерировать неинициализированную матрицу
        explicit BLOCKED_Matrix(ILP_TYPE ctxt, MATRIX_TYPE type, size_t n, size_t m,

        // Распределить матрицу A по блокам
        explicit BLOCKED_Matrix(ILP_TYPE ctxt, MATRIX_TYPE type, const Matrix<T>& A,

        // Make dims for multiply matrix ??????
        explicit BLOCKED_Matrix(const BLOCKED_Matrix<T>& A, const BLOCKED_Matrix<T>&

        // Получить элемент по глобальным индексам i и j. (Требует участия как миним
        T get(size_t i, size_t j) const;
```

```
// Установить элемент по глобальным индексам i и j значению num. (Требует уч
// и того, у кого находится блок с данным элементом)
void set(size_t i, size_t j, T num);

// Важный элемент библиотеки Intel OneAPI, но для пользователя библиотеки QC
std::vector<ILP_TYPE> desc() const;

// Операторные функции. Главная цель - чтобы работать с данным классом было
BLOCKED_Matrix<T> operator*(const BLOCKED_Matrix<T>& B) const;
void operator*=(const BLOCKED_Matrix<T>& B);
BLOCKED_Matrix<T> operator*(T num) const;
void operator*=(T num);
BLOCKED_Matrix<T> operator+(const BLOCKED_Matrix<T>& B) const;
void operator+=(const BLOCKED_Matrix<T>& B);
BLOCKED_Matrix<T> operator+(T num) const;
void operator+=(T num);
BLOCKED_Matrix<T> operator-(const BLOCKED_Matrix<T>& B) const;
void operator-=(const BLOCKED_Matrix<T>& B);
BLOCKED_Matrix<T> operator-(T num) const;
void operator-=(T num);
BLOCKED_Matrix<T> operator/(T num) const;


// ??????????????????????????????
std::vector<T> operator*(const std::vector<T>& A) const;
void operator*=(const std::vector<T>& A);
std::vector<T> operator+(const std::vector<T>& A) const;
void operator+=(const std::vector<T>& A);
std::vector<T> operator-(const std::vector<T>& A) const;
void operator-=(const std::vector<T>& A);

// Вернуть локальный размер общего блока матрицы (число строк)
size_t local_n() const { return local_matrix_.n(); }

// Вернуть локальный размер общего блока матрицы (число столбцов)
size_t local_m() const { return local_matrix_.m(); }

// Вернуть глобальное число строк в матрице
size_t n() const { return n_; }

// Вернуть глобальное число столбцов в матрице
size_t m() const { return m_; }

// Вернуть число строк в подблоках
size_t NB() const { return NB_; }
```

```
// Вернуть число столбцов в подблоках
size_t MB() const { return MB_; }

// Вернуть BLACS context
ILP_TYPE ctxt() const { return ctxt_; }

// Вернуть тип матрицы
MATRIX_TYPE matrix_type() const { return matrix_type_; }

// Вернуть весь общий блок в виде одномерного массива
Matrix<T>& get_local_matrix() { return local_matrix_;}
const Matrix<T>& get_local_matrix() const { return local_matrix_;}

// Вернуть указатель на локальный элемент по локальным индексам i и j.
T* data(size_t i = 0, size_t j = 0) { return local_matrix_.data() + get_loca
const T* data(size_t i = 0, size_t j = 0) const { return local_matrix_.data(

// Вернуть локальный элемент по локальный индексам i и j.
T& operator()(size_t i, size_t j) { return local_matrix_(i, j); }
const T operator()(size_t i, size_t j) const { return local_matrix_(i, j); }

// Распечатать матрицу в блочной виде, то есть каждый процесс напечатает сво
void print_distributed(const std::string& name) const;

// Распечатать матрицу в виде обычной матрицы
void show(size_t width = QConfig::instance().width(), ILP_TYPE root_id = mpi

// Записать матрицу в файл формата CSV
void write_to_csv_file(const std::string& filename) const;

// Получить глобальный номер измерения по локальному
ILP_TYPE get_global_row(size_t i) const;
ILP_TYPE get_global_col(size_t j) const;

// Получить локальный номер измерения по глобальному
ILP_TYPE get_local_row(size_t i) const;
ILP_TYPE get_local_col(size_t j) const;

// Получить строку, в которой находится процессор, которому принадлежит стро
ILP_TYPE get_row_proc(size_t i) const;

// Получить столбец, в которой находится процессор, которому принадлежит сто
ILP_TYPE get_col_proc(size_t j) const;

// Сделать эрмитовое преобразование матрице
BLOCKED_Matrix<T> hermit() const;
```

```
        // Получить весь столбец целиком
        std::vector<T> col(size_t i) const;

        // Получить всю строку целиком
        std::vector<T> row(size_t j) const;
    protected:
        size_t get_global_index(size_t i, size_t j) const { return j * n_ + i; }
        size_t get_local_index(size_t i, size_t j) const { return j * local_matrix_.

        MATRIX_TYPE matrix_type_; // Тип матрицы
        ILP_TYPE ctxt_; // BLACS context
        size_t n_; // Глобальное число строк матрицы
        size_t m_; // Глобальное число столбцов в матрице
        size_t NB_; // Число строк в подблоке
        size_t MB_; // Число столбцов в подблоке
        Matrix<T> local_matrix_; // Локальный общий блок матрицы
};
```

2.3 BLOCKED_Vector<T>

Блочнораспределённый вектор. Является наследником BLOCKED_Matrix<T>. Про NB смотреть в разделе про BLOCKED_Matrix<T>.

```
template<typename T>
class BLOCKED_Vector: public BLOCKED_Matrix<T> {
    public:
        explicit BLOCKED_Vector() = default;

        // Распределяет вектор между процессорами (копия вектора x есть у всех)
        explicit BLOCKED_Vector(ILP_TYPE ctxt, const std::vector<T>& x);

        // ??????????????????
        explicit BLOCKED_Vector(const BLOCKED_Vector<T>& x, const Matrix<T>& local_v

        // Создать вектор из функции размера n.
        explicit BLOCKED_Vector(ILP_TYPE ctxt, size_t n, std::function<T(size_t, siz

        // Создать вектор из функции размера n с начальным значением value
        explicit BLOCKED_Vector(ILP_TYPE ctxt, size_t n, T value, size_t NB = 0);

        // Создать неинициализированный вектор размера n
        explicit BLOCKED_Vector(ILP_TYPE ctxt, size_t n, size_t NB = 0);

        // Распределить вектор, хранящийся на ядре по номером root_id вектор x по вс
        explicit BLOCKED_Vector(ILP_TYPE ctxt, const std::vector<T>& x, ILP_TYPE roo
```

```
        // Сделать по размерностям результата умножения ????????????????
        explicit BLOCKED_Vector(const BLOCKED_Matrix<T>& A, const BLOCKED_Vector<T>&

        // Получить элемент i (выдаёт копию, не ссылку)
        T get(size_t i) const { return this->BLOCKED_Matrix<T>::get(i, 0); }

        // Установить элемент i
        void set(size_t i, T num) { this->BLOCKED_Matrix<T>::set(i, 0, num); }

        // Получить элемент i (даёт ссылку)
        T& operator[](size_t i) { return BLOCKED_Matrix<T>::local_matrix_(i, 0); }
        const T& operator[](size_t i) const { return BLOCKED_Matrix<T>::local_matrix

        // Второй способ получения элемента i
        T& operator()(size_t i) { return BLOCKED_Matrix<T>::local_matrix_(i, 0); }
        const T& operator()(size_t i) const { return BLOCKED_Matrix<T>::local_matrix

        // Операции с вектором
        BLOCKED_Vector<T> operator*(const BLOCKED_Matrix<T>& A) const;
        BLOCKED_Vector<T> operator+(const BLOCKED_Vector<T>& x) const;
        void operator+=(const BLOCKED_Vector<T>& x);
        BLOCKED_Vector<T> operator-(const BLOCKED_Vector<T>& x) const;
        void operator-=(const BLOCKED_Vector<T>& x);
        BLOCKED_Vector<T> operator*(const BLOCKED_Vector<T>& x) const;
        BLOCKED_Vector<T> operator/(const BLOCKED_Vector<T>& x) const;

        // Получить весь вектор целиком на все процессы
        std::vector<T> get_vector() const;

        BLOCKED_Vector<T> operator+(T x) const;
        BLOCKED_Vector<T> operator-(T x) const;
        BLOCKED_Vector<T> operator*(T x) const;
        BLOCKED_Vector<T> operator/(T x) const;
};
```

2.4 Вспомогательные функции

Как было сказано в введении, версия CLUSTER является расширением одноядерной версии. Некоторые алгоритмы плохо параллелится напрямую с помощью блочного распределения, например гамильтонианы маленького размера. Их вообще лучше блочно не распределять. Вместо этого, можно попытаться распределять одноядерные версии напрямую. В директории examples лежит подобный пример - <вставить имя программы>.

Здесь приведены вспомогательные функции, как для ручного распараллеливания программы, так и для других целей. Реализации лежат в файлах functions.*, если не сказано иное.

make_rank_map - распределение ядер по вектору

Самое частая ситуация, которая возникала на практике. У нас есть вектор, например, с какими-нибудь параметрами для гамильтониана. Мы хотим, чтобы ядра взяли на себя равномерную нагрузку, проходя по этому вектору. То есть, допустим у нас вектор размера n, ядер - p. Тогда каждое ядро должно пройтись по $\frac{n}{p}$ элементам.

Данная функция собственно это и делает. Вместо параметро гамильтонианов может быть что угодно, например столбцы матрицы. Дальше только Ваша фантазия.

```
// size - размер вектора. rank - номер ядра процессора. world_size - общее число яде
// start_col и count - переменные, в которые записывается результат. start_col - ном
// с которого ядро начинает, count - сколько элементов оно должно пройти.

void make_rank_map(size_t size, int rank, int world_size, size_t& start_col, size_t&
```

Пример использования:

```
<<вставить пример>>
```

init_grid, init_vector_grid, blacs_gridinfo - инициализация решёток BLACS

Реализация в файлах mpi_functions.*

```
// ctxt - возвращаемое значение (контекст)
// proc_rows - число строк в решётке процессоров
// proc_cols - число столбцов в решётке процессоров
// 0 - автоматический выбор значения (рекомендуется)

// Пытается подогнать под квадратную решётку
void init_grid(int& ctxt, ILP_TYPE proc_rows = 0, ILP_TYPE proc_cols = 0);

// Выстраивает процессы в ряд
void init_vector_grid(int& ctxt, ILP_TYPE proc_rows = 0, ILP_TYPE proc_cols = 0);

// myrow, mycol - координаты ядра в решётке
// Получить информацию о решётке
void blacs_gridinfo(const ILP_TYPE& ctxt, ILP_TYPE& proc_rows, ILP_TYPE& proc_cols,
```

blocked_matrix_get_col, blocked_matrix_get_row, blocked_matrix_to_blocked_ve

В какой-то момент может возникнуть такое желание, получить столбец/строку блочно-распределённой матрицы в виде блочного вектора, но уже с иным распределением. Для такой задачи придётся создавать новый контекст, об этом подробнее написано в инструкции. blocked_matrix_to_blocked_vectors делает же тоже самое, что и blocked_matrix_get_col, но для всей матрицы в целом, сохраняя блочные вектора в вектор.

```
// ctxt - новый контекст, согласно которому идёт распределение. То есть ядра будут р

BLOCKED_Vector<double> blocked_matrix_get_col(ILP_TYPE ctxt, const BLOCKED_Matrix<do
BLOCKED_Vector<double> blocked_matrix_get_row(ILP_TYPE ctxt, const BLOCKED_Matrix<do
BLOCKED_Vector<COMPLEX> blocked_matrix_get_col(ILP_TYPE ctxt, const BLOCKED_Matrix<C
BLOCKED_Vector<COMPLEX> blocked_matrix_get_row(ILP_TYPE ctxt, const BLOCKED_Matrix<C
std::vector<BLOCKED_Vector<double>> blocked_matrix_to_blocked_vectors(ILP_TYPE ctxt,
```

```
std::vector<BLOCKED_Vector<COMPLEX>> blocked_matrix_to_blocked_vectors(ILP_TYPE ctxt
```

На данный момент, это все вспомогательная функция для ручного распределения одноядерных гамильтонианов и операторов. Далее идут вспомогательные функции для других целей.

linspace - создать вектор времени (полностью аналогичен linspace в Python)

Создаёт, собственно, вектор времени. На практике он маленького размера, особенно в сравнении с остальными абстракциями, поэтому распределять его нет никакого смысла. Замечу, что результат всегда std::vector<double>.

```
// start_in - начало отрезка
// end_in - конец отрезка
// num_in - количество чисел в отрезке

template<typename T>
std::vector<T> linspace(T start_in, T end_in, int num_in)
```

Пример использования:

```
<<вставить пример>>
```

E_Matrix - создать единичную матрицу размера n

```
// n - размер матрицы

template<typename T>
Matrix<T> E_Matrix(size_t n);
```

get_elem_from_set - получить элемент из set по индексу.

Причина данного костыля весьма банальна. Нумерация столбцов и строк в гамильтониане напрямую завязана на базисе, он же в свою очередь как-то отсортирован, и данная сортировка ВЕЗДЕ должна быть одинакова. Собственно, базис хранится в std::set. Иногда получается так, что хочется взять элемент из базиса под определённым номером или выше определённого (или ниже), в общем, нужна индексация. Базисы бывают огромные, и для того, чтобы не перебирать все, отсекая не нужные, и была сделана эта функция.

```
// st - множество элементов
// index - индекс элемента в множестве

T get_elem_from_set(const std::set<T>& st, size_t index);
```

set_query??????????????? - отсекает из множества элементы, не удовлетворяющие булевой функции

```
// v - множество
// func - булевая функция

std::set<T> set_query(const std::set<T>& v, const std::function<bool(T)>& func);
```

scalar_product - скалярное произведение векторов.

У нас 2 вида векторов - обычные и блочно распределённые. Блочно распределённые реализованы в blocked_vector.*.

```
// <b|a>, (a, b)
double scalar_product(const std::vector<double>& a, const std::vector<double>& b);
COMPLEX scalar_product(const std::vector<COMPLEX>& a, const std::vector<COMPLEX>& b)
double scalar_product(const BLOCKED_Vector<double>& a, const BLOCKED_Vector<double>&
COMPLEX scalar_product(const BLOCKED_Vector<COMPLEX>& a, const BLOCKED_Vector<COMPLE
```

fmin - найти минимум функции (Ищет локальный минимум)

```
// f must be unimodal on [a, b]
double fmin(std::function<double(double)> f, double a, double b, double eps = QConfi
```

fsolve - решает f(x) = target

```
// Solve f(x) = target
double fsolve(std::function<double(double)> f, double a, double b, double target = 0
```

Cubic_Spline_Interpolate - интерполяция кубическими сплайнами. Возвращает функцию

```
std::function<double(double)> Cubic_Spline_Interpolate(const std::vector<double>& x,
```

is_zero - проверяет, близко ли число к нулю. Регулируется с помощью QConfig

```
bool is_zero(double a, double eps = QConfig::instance().eps());
bool is_zero(COMPLEX a, double eps = QConfig::instance().eps());
```

is_in_vector - проверить, есть ли элемент в векторе

```
template<typename T>
bool is_in_vector(const std::vector<T> v, const T& elem);
```

Pro_Race_Algorithm????????RENAME - решить СЛАУ методом трёхдиагональной прогонки

```
template<typename T>
std::vector<T> Pro_Race_Algorithm(const Matrix<T>& B, const std::vector<T>& y);
```

convert_to - конвертировать множество векторов от StateType к Basis_State

```
//StateType - дочерний класс от Basis_State
//states - множество состояний

template<typename StateType>
std::set<Basis_State> convert_to(const std::set<StateType>& states);
```

f_vector - from x1, x2, x3 ... to f(x1), f(x2), ...

```
template<typename T>
std::vector<T> f_vector(std::function<T(T)> f, const std::vector<T>& x);
```

2.5 Понятие БАЗОВОГО состояния Basis_State

Основа всей библиотеки. Первая версия отталкивалась от понятия оператора, но на практике оно оказалось крайне непрактичным. Данная версия берёт за основу понятие состояния. Как было видно на схеме, из Basis_State мы делаем свой StateType, а дальше всё исходит напрямую из него.

```
class Basis_State {
    public:
        // инициализация пустого состояния
        explicit Basis_State() = default;

        // groups_count делит кудиты на равные по размеру группы. max_val - максимал
        explicit Basis_State(size_t qudits_count, ValType max_val = 1, size_t groups

        // добавление поддержки для разных кудитов. max_vals - вектор, в котором для
        explicit Basis_State(size_t qudits_count, const std::vector<ValType>& max_va
                             size_t groups_count = 1);

        // qudit_count - чилсо кудитов. max_val - максимальное значение их всех. gro
        // В сумме должно получиться qudits_count, иначе ошибка.
        // ВНИМАНИЕ!!! - groups потом хранится в другом виде. А именно, в нём хранят
        // Например, для |10>|001> вектор groups_ = {1, 4}
        // Для ориентации в данном векторе сделаны отдельные методы, приведённые ниж
        explicit Basis_State(size_t qudits_count, ValType max_val, const std::vector

        // ====== Далее идут варианты с инициализацией значений ========

        // Создания состосния с инициализацией значений
        explicit Basis_State(const std::vector<ValType>& qudits, ValType max_vals =

        // Создание состояния с инициализация значений с поддержкой разных кудитов
        explicit Basis_State(const std::vector<ValType>& qudits, const std::vector<V
                             size_t groups_count = 1);

        // Создание состояния с инициализация значений с поддержкой разных кудитов +
        explicit Basis_State(const std::vector<ValType>& qudits,  const std::vector<
                             const std::vector<size_t>& groups_sizes);

        // Установить кудиту значение. qudit_index - индекс кудита. group_id - номер
        void set_qudit(ValType val, size_t qudit_index, size_t group_id = 0);
```

```
// Получить значение кудита
ValType get_qudit(size_t qudit_index, size_t group_id = 0);

// Добавить кудит. Будет добавлен к последней группе
void append_qudit(ValType init_val = 0, ValType max_val = 1);

// Проверка, является ли состояние пустым
bool is_empty();

// Получить дополнительную информацию о состоянии
std::string get_info();
// Записать дополнительную информацию о состоянии
void set_info(const std::string& str);


// Получить индексы последних кудитов групп
std::vector<size_t> get_groups();
// Получить индекс начала группы
size_t get_group_start(size_t group_id);
// Получить индекс конца группы (Например |10>|10> для 0 группы индекс конца
size_t get_group_end(size_t group_id);
// Получить число групп
size_t get_groups_count();
// Получить размер  группы
size_t get_group_size(size_t group_id);

// Получить в качестве состояния отдельную группу
Basis_State get_group(size_t group_id) const;

// Конвертировать состояние в строку
std::string to_string() const;
// Выдаст ошибку, если состояния различаются по группам или по максимальным
bool operator==(const Basis_State& other);
bool operator<(const Basis_State& other);

// Поменять максимальное значение кудита
void set_max_val(ValType val, size_t qudit_index, size_t group_id = 0);
// Получить максимальное значение кудита
ValType get_max_val(size_t qudit_index, size_t group_id = 0);
// Получить вектор максимальных значений
std::vector<ValType> max_vals();

// Получить индекс состояния в базисе
size_t get_index(const std::set<Basis_State>& basis) const;

// Очистить состояние
```

```
        void clear() { qudits_.resize(0); max_vals_.resize(0); groups_.resize(0); }
    protected:
        std::string info_; // Дополнительная информация о состоянии
        std::vector<ValType> qudits_; // Вектор кудитов
        std::vector<ValType> max_vals_; // Вектор максимальных значений кудитов
        std::vector<size_t> groups_; // Вектор начала групп
};
```

2.6 Понятие состояния State<StateType>

Basis_State это базисное состояние, в нём нет никакой суперпозиции. StateType - аналогично. State<StateType> же представляет собой привычное понятие состояния $|\Psi\rangle$

```
template<typename StateType>
class State {
    public:
        // Инициализировать пустое состояние
        explicit State() = default;
        // Конструктор копирования
        State(const State<StateType>& state) = default;
        // Привести базисное состояние к обычному
        State(const StateType& state);

        // ??????????????????????????????????????????????????????????????
        explicit State(const std::string& state_string, ValType max_val = 1);
        explicit State(const std::string& state_string, const std::vector<ValType>& 

        // Умножить состояние на коэффициент c
        State<StateType> operator*(const COMPLEX& c);

        // Сложить 2 вектора состояния. Если состояния нет в базисе - добавляется, и
        void operator+=(const State<StateType>& st);

        // Есть ли базисное состояние в нашем состоянии
        bool is_in_state(const StateType& state);

        // Получить амплитуду состояния под индексом index
        COMPLEX& operator[](size_t index);
        COMPLEX operator[](size_t index);

        // Общее число состояний
        size_t size() const { return state_vec_.size(); }

        // Получить индекс базисного состояния в базисе
        size_t get_index(const StateType& state);
```

```
        // Вставить базисное состояние с амплитудой amplitude
        void insert(const StateType& state, const COMPLEX& amplitude = COMPLEX(0, 0)

        // Установить базис
        void set_state_components(const std::set<StateType>& st);
        // Установить вектор амплитуд
        void set_vector(const std::vector<COMPLEX>& v);
        // Получить базис
        std::set<StateType> get_state_components() const;
        // Получить вектор амплитуд
        std::vector<COMPLEX> get_vector() const;

        // Привести состояние к строковому виду
        std::string to_string() const;

        // ИСПРАВИТЬ В DYNAMIC!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
        // Используется при моделировании. Гамильтониан хранит std::set<Basis_State>
        // Базис генерируется во время генерации гамильтониана, чтобы подогнать его
        // и был сделан данный метод
        State<Basis_State> fit_to_basis(const std::set<Basis_State>& basis) const;
    private:
        std::vector<COMPLEX> state_vec_; // Вектор амплитуд
        std::set<StateType> state_components_; // Базис состояния
};
```

2.7 Понятие оператора Operator<T>.

2.8 Понятие гамильтониана.

2.9 Моделирование динамики гамильтонианов.

3 Визуализация